Initial raw data clean-up

Overview

This notebook describes my tidying of the raw data from the fall 2026 CURIOSS OSPO survey. Mostly I'm just taking the data out of excel and writing it to tab-separated files for easy downstream analysis with r. I'm writing them to my data/ folder in the root of this project.

These data could go in a relational database, but why overcomplicate things? tsv files will do the job.

Import packages, utilities, and config variables

```
library(readxl)
library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':

    filter, lag
```

```
The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(docstring)
```

```
Attaching package: 'docstring'

The following object is masked from 'package:utils':

    ?
```

```
library(readr)
```

```
Warning: package 'readr' was built under R version 4.4.3
```

```
library(stringr)

source("utils.R")

# Read config.yml
cfg <- config::get()

# Read the external mappings CSV
# Expected columns: category, nickname, original_name
mappings <- read_tsv(file.path(cfg$data_path, "mappings.tsv"), show_col_types = FALSE)
```

Define functions

```
remove_uninformative_rows <- function(df) {
  #' Remove Uninformative Rows
  #'
  #' Removes rows that contain either an NA object, an empty string, or a
  #' whitespace string ("  ") in every column.
  #'
  #' @param df Data frame to process.
  #' @return A copy of the data frame with uninformative rows removed.
  df |>
    filter(if_any(everything(), function(x) {
      val <- trimws(x) # trim whitespace
```

```
      !is.na(val) & val != ""
    }))
}

clean_p_tags <- function(df) {
  #' Clean HTML P Tags from Data Frame Entries
  #'
  #' Removes opening `<p>` (including any HTML attributes) and closing `</p>` tags
  #' from the start and end of all character or factor columns in a data frame.
  #'
  #' @param df A data frame containing character or factor columns to clean.
  #'
  #' @return A data frame with outer `<p>` and `</p>` tags stripped from all string entries.
  #'
  #' @examples
  #' df <- data.frame(
  #'   title = c("<p>First Title</p>", "Second Title"),
  #'   description = c("<p class='intro'>Body text</p>", "No tags")
  #' )
  #' clean_p_tags(df)

  df[] <- lapply(df, function(col) {
    if (is.character(col) || is.factor(col)) {
      # Remove leading <p> (with optional attributes/whitespace) and trailing </p>
      col <- gsub("^\\s*<p[^>]*>", "", col, ignore.case = TRUE)
      col <- gsub("</p>\\s*$", "", col, ignore.case = TRUE)
    }
    return(col)
  })
  return(df)
}

select_and_rename <- function(df, name_map) {
  #' Select and Rename Columns Using a Named Vector
  #'
  #' @description
  #' Extracts a subset of columns from a data frame based on the values in a
  #' named vector and renames them to match the corresponding vector names.
  result <- df |>
    select(any_of(unname(name_map))) |>
    rename(any_of(name_map))
```

```
  return(result)
}

get_mapping <- function(map_category, mapping_df) {
  #' Generate a Named Vector from the Mappings Dataframe
  #'
  #' @param map_category Character string matching the 'category' column.
  #' @param mapping_df The dataframe containing all mappings.
  #' @return A named character vector.
  subset_df <- mapping_df |> filter(category == map_category)
  setNames(subset_df$original_name, subset_df$nickname)
}
```

Read data from excel

After receiving the data from Clare, a password-protected .xlsx file, I copied the data into a new excel file without password protection so R's readxl package could read it. That file is called raw_data.xlsx.

```
data <- readxl::read_excel("data/raw_data.xlsx", col_types = "text", na = c(""))
# Using a workaround to preserve distinction between
# "" and "N/A", because by default read_excel converts
# both of these to NA objects.
data[is.na(data)] <- ""
data[data == "N/A"] <- NA

# Remove Windows line endings (\r\n), Unix line endings (\n), and carriage returns (\r).
data <- data |>
  mutate(across(where(is.character), ~ str_replace_all(.x, "[\r\n]+", " ")))
```

Initial exploration and clean-up

```
dim(data)
```

```
[1]  36 198
```

```
names(data)[1:15]
```

```
 [1] "Response ID"
 [2] "Date submitted"
 [3] "Last page"
 [4] "Start language"
 [5] "Seed"
 [6] "Quota exit"
 [7] "Your organization / institution name"
 [8] "Your email"
 [9] "In which region is your organization located?"
[10] "In which region is your organization located? [Other]"
[11] "Which best describes the open source function at your organization?"
[12] "Which best describes the open source function at your organization? [Other]"
[13] "Where is your OSPO located within your university? [University library]"
[14] "Where is your OSPO located within your university? [Central administration (e.g., centr
[15] "Where is your OSPO located within your university? [Technology transfer, innovation, or
```

Remove uninformative columns

```
cols_to_remove <- c(
    "Response ID",
    "Date submitted",
    "Last page",
    "Start language",
    "Seed",
    "Quota exit")

data <- data |> select(-any_of(cols_to_remove))
dim(data)
```

```
[1]  36 192
```

Remove uninformative rows

```
data <- remove_uninformative_rows(data)
dim(data)
```

```
[1]  26 192
```

Remove the very first response, which I suspect was a test by one of the survey organizers. The organization is listed as "dsad" and the email is "dsas", and the locations of those keys on a qwerty keyboard support my hypothesis that that was just some nonsense for a test run. The only question that that person answered was the "choose your top 3" drag-and-drop, so I assume an organizer was just testing that that question worked.

```
data <- data |> slice(-1)
```

Remove HTML

tags that are present in some of the responses (presumably a bug from limesurvey).

```
data <- clean_p_tags(data)
```

Simple multiple-choice

I'll start with the simple multiple choice questions. I'll dump all of these in a single file called mc_simple.tsv.

```
mc_simple_names_map <- get_mapping("mc_simple", mappings)
mc_simple <- select_and_rename(data, mc_simple_names_map)
write_df_to_file(mc_simple, 'mc_simple.tsv')
```

Complex multiple-choice

Now let's create a couple more files for the multiple choice questions that had an "other" write-in option. mc_complex.tsv will have just the multiple-choice part of these "complex" multiple choice questions. mc_other.tsv has the corresponding "other" responses where people wrote-in additional info.

```
mc_complex_names_map <- get_mapping("mc_complex", mappings)
mc_other_names_map <- get_mapping("mc_other", mappings)

mc_complex <- select_and_rename(data, mc_complex_names_map)
mc_other <- select_and_rename(data, mc_other_names_map)

write_df_to_file(mc_complex, 'mc_complex.tsv')
write_df_to_file(mc_other, 'mc_other.tsv')
```

Qualitative responses

Let's put the qualitative responses (not associated with a multiple-choice (MC) or select-all-that-apply (SATA) question) in their own file. I like to split obvious personally identifiable information (PII) into its own file, just to mitigate the potential bias introduced by looking at it while I'm reading the responses.

```
qual_pii_names_map <- get_mapping("qual_pii", mappings)
qual_pii <- select_and_rename(data, qual_pii_names_map)
write_df_to_file(qual_pii, 'qual_pii.tsv')

qual_names_map <- get_mapping("qual", mappings)
qual <- select_and_rename(data, qual_names_map)
write_df_to_file(qual, 'qual.tsv')
```

Matrix questions

Now let's put the matrix questions in their own files. Since these questions are complex, I'm going to give each question its own file.

Just like the "complex" multiple-choice questions, these questions also have an "other" write-in option, so I will put these in a separate and corresponding file.

```
objectives_name_map <- get_mapping("objectives", mappings)
objectives <- select_and_rename(data, objectives_name_map)
write_df_to_file(objectives, 'objectives.tsv')

blockers_names_map <- get_mapping("blockers", mappings)
blockers <- select_and_rename(data, blockers_names_map)
write_df_to_file(blockers, 'blockers.tsv')

matrix_other_names_map <- get_mapping("matrix_other", mappings)
matrix_other <- select_and_rename(data, matrix_other_names_map)
write_df_to_file(matrix_other, 'matrix_other.tsv')
```

Ranking question

I think this question merits its own file: "Of the activities above, which take the most of your own time? Rank your top three."

```
ranking_name_map <- get_mapping("ranking", mappings)
ranking <- select_and_rename(data, ranking_name_map)
write_df_to_file(ranking, 'ranking.tsv')
```

SATA questions

There were a lot of SATA (Select All That Apply) questions, each with an "other" option. Individually, they are not very complicated, but together, they are complex enough that I'd like to put them in separate files.

I'll put all the "other" responses to these SATA questions in a single file called sata_other.tsv.

OSPO location

```
ospo_location_names_map <- get_mapping("ospo_location", mappings)
ospo_location <- select_and_rename(data, ospo_location_names_map)
write_df_to_file(ospo_location, 'ospo_location.tsv')

ospo_location_other_names_map <- get_mapping("ospo_location_other", mappings)
ospo_location_other <- select_and_rename(data, ospo_location_other_names_map)

sata_other <- ospo_location_other
```

Professional background

```
prof_bg_names_map <- get_mapping("prof_bg", mappings)
prof_bg <- select_and_rename(data, prof_bg_names_map)
write_df_to_file(prof_bg, 'prof_bg.tsv')

prof_bg_other_names_map <- get_mapping("prof_bg_other", mappings)
prof_bg_other <- select_and_rename(data, prof_bg_other_names_map)

sata_other <- cbind(sata_other, prof_bg_other)
```

OSPO funding

```
ospo_funding_names_map <- get_mapping("ospo_funding", mappings)
ospo_funding <- select_and_rename(data, ospo_funding_names_map)
write_df_to_file(ospo_funding, 'ospo_funding.tsv')

ospo_funding_other_names_map <- get_mapping("ospo_funding_other", mappings)
ospo_funding_other <- select_and_rename(data, ospo_funding_other_names_map)

sata_other <- cbind(sata_other, ospo_funding_other)
```

Best practices

```
best_practices_names_map <- get_mapping("best_practices", mappings)
best_practices <- select_and_rename(data, best_practices_names_map)
write_df_to_file(best_practices, 'best_practices.tsv')

best_practices_other_names_map <- get_mapping("best_practices_other", mappings)
best_practices_other <- select_and_rename(data, best_practices_other_names_map)

sata_other <- cbind(sata_other, best_practices_other)
```

Educational efforts

```
edu_efforts_names_map <- get_mapping("edu_efforts", mappings)
edu_efforts <- select_and_rename(data, edu_efforts_names_map)
write_df_to_file(edu_efforts, 'edu_efforts.tsv')

edu_efforts_other_names_map <- get_mapping("edu_efforts_other", mappings)
edu_efforts_other <- select_and_rename(data, edu_efforts_other_names_map)

sata_other <- cbind(sata_other, edu_efforts_other)
```

External partnerships and tech transfer

```
ext_partnerships_names_map <- get_mapping("ext_partnerships", mappings)
ext_partnerships <- select_and_rename(data, ext_partnerships_names_map)
write_df_to_file(ext_partnerships, 'ext_partnerships.tsv')

ext_partnerships_other_names_map <- get_mapping("ext_partnerships_other", mappings)
ext_partnerships_other <- select_and_rename(data, ext_partnerships_other_names_map)

sata_other <- cbind(sata_other, ext_partnerships_other)
```

Supporting tools and infrastructure

```
tools_infra_names_map <- get_mapping("tools_infra", mappings)
tools_infra <- select_and_rename(data, tools_infra_names_map)
write_df_to_file(tools_infra, 'tools_infra.tsv')

tools_infra_other_names_map <- get_mapping("tools_infra_other", mappings)
tools_infra_other <- select_and_rename(data, tools_infra_other_names_map)

sata_other <- cbind(sata_other, tools_infra_other)
```

Community building

```
comm_building_names_map <- get_mapping("comm_building", mappings)
comm_building <- select_and_rename(data, comm_building_names_map)
write_df_to_file(comm_building, 'comm_building.tsv')

comm_building_other_names_map <- get_mapping("comm_building_other", mappings)
comm_building_other <- select_and_rename(data, comm_building_other_names_map)

sata_other <- cbind(sata_other, comm_building_other)
```

Directly supporting open source projects

```
project_support_names_map <- get_mapping("project_support", mappings)
project_support <- select_and_rename(data, project_support_names_map)
write_df_to_file(project_support, 'project_support.tsv')
```

```
project_support_other_names_map <- get_mapping("project_support_other", mappings)
project_support_other <- select_and_rename(data, project_support_other_names_map)

sata_other <- cbind(sata_other, project_support_other)
```

Advocacy and policy

```
advocacy_policy_names_map <- get_mapping("advocacy_policy", mappings)
advocacy_policy <- select_and_rename(data, advocacy_policy_names_map)
write_df_to_file(advocacy_policy, 'advocacy_policy.tsv')

advocacy_policy_other_names_map <- get_mapping("advocacy_policy_other", mappings)
advocacy_policy_other <- select_and_rename(data, advocacy_policy_other_names_map)

sata_other <- cbind(sata_other, advocacy_policy_other)
```

Funding and financial support

```
funding_support_names_map <- get_mapping("funding_support", mappings)
funding_support <- select_and_rename(data, funding_support_names_map)
write_df_to_file(funding_support, 'funding_support.tsv')

funding_support_other_names_map <- get_mapping("funding_support_other", mappings)
funding_support_other <- select_and_rename(data, funding_support_other_names_map)

sata_other <- cbind(sata_other, funding_support_other)
```

OSPO operations and strategy

```
ospo_ops_names_map <- get_mapping("ospo_ops", mappings)
ospo_ops <- select_and_rename(data, ospo_ops_names_map)
write_df_to_file(ospo_ops, 'ospo_ops.tsv')

ospo_ops_other_names_map <- get_mapping("ospo_ops_other", mappings)
ospo_ops_other <- select_and_rename(data, ospo_ops_other_names_map)
```

```
sata_other <- cbind(sata_other, ospo_ops_other)
```

Write SATA 'other' responses to a single file

```
write_df_to_file(sata_other, 'sata_other.tsv')
```